

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_634e841e-51b\agent-ab9f2f22e8b2d66fe.jsonl`  
- SHA-256: `4b935e7b90621ffd5c8e328c1362d12028a43295594d7fa989156bb844983f21`  
- Source modified: `2026-07-06T08:41:24+00:00`  
- Imported at: `2026-07-08T16:00:06+00:00`  
- project: `wf_634e841e-51b`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T08:39:27.317Z)

Reviewing GitHub PR #38 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-d-command-service -> main, one commit 22abe72).
It implements ADR 0011 Track D: a REFACTOR that extracts command parsing + config-mutation logic from the
Telethon adapter into a provider-independent CommandService. THE PRIMARY REVIEW RISK IS BEHAVIORAL
EQUIVALENCE: the new code must behave identically to the old inline logic for every command.

Changed files ONLY: src/tg_video_filter/command_service.py (NEW),
src/tg_video_filter/commands.py (refactored
407->~120 line thin Telethon adapter), tests/test_command_service.py (NEW, 20 tests). Diff
+650/-376.
tests/test_commands.py is UNCHANGED and still passes (backwards-compat surface).

To compare against the pre-refactor behaviour, read the OLD file:
git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/commands.py
The new logic lives in command_service.py (CommandService.handle + parsers + formatters) and the new
commands.py delegates to it. Key already-checked facts (verify independently, don't just trust):
- regex patterns are byte-identical old vs new; branch order is the same.
- old handler did text.rstrip() then per-branch logic; new handler passes raw text and handle() does text.rstrip().
- old _update_config RELOADED cfg from db each call; new CommandService._update uses the passed current_cfg,
and the new adapter loads current_cfg = db.load_filter_config(...) fresh right before handle(). Assess equivalence.
- both old and new use cfg.model_copy(update=...) which SKIPS pydantic validation (pre-existing, not new).

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
- A STRONG equivalence check: feed a battery of command strings through BOTH the new CommandService.handle() and (on a checkout of main, or by reasoning from the old source) the old handler, and diff reply_text + resulting saved FilterConfig. Report ANY divergence (different reply string, different config mutation, different validation threshold, different branch taken, different order).
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check clean; ruff format clean; pytest
482 passed @ 97.05% (floor 80%, up from 96.52%).
- Report ONLY defects in THIS PR's diff. For a refactor, a BEHAVIORAL DIVERGENCE from the old

code is the

highest-value finding. Also flag: lost functionality, broken re-exports, new bugs in extracted logic.

No praise. Concrete failure scenario per finding. Rank most-severe first. Honest severity.

- Severity: blocker (wrong behaviour users hit / lost command), high (real divergence, narrow trigger),

medium (edge divergence or latent), low (hygiene/style).

DIMENSION: BEHAVIORAL EQUIVALENCE (old commands.py handler vs new CommandService.handle + new adapter).

Do a DIFFERENTIAL analysis of EVERY command branch: /set size, /set length (& /set max length), /set min length,

/set min resolution (& min res), /set exclude gif|round|nosound on|off, /show config, /show commands, the

help/malformed fallbacks, and unrecognized noise. For each: same regex, same value parsing, same validation

threshold, same error/success reply STRING, same config field mutated, same branch order, same text.rstrip

handling. Also compare: the current_cfg-vs-DB-reload difference (old _update_config reloaded; new uses passed

current_cfg + adapter loads fresh); the engine rebuild (old built FilterEngine from returned cfg; new reloads

from db then builds); the message.out/peer_id gate; _reply target + exception set. STRONGLY PREFERRED: run a

battery repro of command strings through

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe using

command_service.CommandService().handle(...) on an in-memory

db (db._apply_schema; seed account+default policy via db.save_filter_config) and confirm the mutations/replies

match the old semantics. Report ANY divergence. If fully equivalent, say so explicitly (return no findings).

2. assistant (2026-07-06T08:39:29.795Z)

I'll analyze this refactor for behavioral equivalence. Let me start by reading both the old and new code.

3. user (2026-07-06T08:39:31.391Z)

```
"""Config commands: /set size, /set length, /show config.
```

These commands are sent to the user's own Saved Messages (like /backfill) and update the persisted ``FilterConfig``. The filter engine is hot-reloaded so the new config takes effect immediately without a restart.

See ``docs/adr/0007-inline-config-commands.md`` for the design rationale.
"""

```
from __future__ import annotations
```

```
import logging
```

```
import re
```

```
from collections.abc import Callable
```

```
import aiosqlite
```

```
from telethon import TelegramClient
```

```
from telethon.errors import RPCError
```

```
from tg_video_filter import db
```

```
from tg_video_filter.filters import FilterEngine
```

```
from tg_video_filter.models import FilterConfig, User
```

```
log = logging.getLogger(__name__)
```

Key into the shared engine_holder dict. Dict swap is atomic under the GIL for single-threaded asyncio dispatch (see ADR 0007).

```
ENGINE_KEY = "engine"
```

--- regex patterns -----

```
_SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmg]b?)$", re.IGNORECASE)
```

Matches both "/set length" and "/set max length" (legacy alias).

Kept separate from _MIN_LENGTH_RE for clarity; patterns are mutually exclusive.

```
_LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
```

```
_MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
```

```
_MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$", re.IGNORECASE)
```

```
_SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
```

```
_SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
```

```
_EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$", re.IGNORECASE)
```

Catch-all: matches any "/set exclude ..." that didn't match the strict pattern, so we can reply with a friendly usage message instead of silence.

```
_EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
```

Catch-all for malformed /set min resolution|res (same pattern as _EXCLUDE_HELP_RE)

```
_MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
```

--- unit parsing -----

```
def parse_size_mb(value_str: str, unit: str) -> int:
    """Parse a size value + unit string into mebibytes.
```

```
>>> parse_size_mb("10", "MB")
```

```
10
```

```
>>> parse_size_mb("1", "GB")
```

```
1024
```

```
>>> parse_size_mb("500", "KB")
```

```
1
```

```
Note: returns the raw arithmetic result; callers must enforce
``FilterConfig`` domain constraints (e.g. ``max_size_mb >= 1``).
"""
```

```
value = int(value_str)
```

```
unit = unit.lower().rstrip("b") # "MB" ? "m", "KB" ? "k", "GB" ? "g"
```

```
multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
```

```
if unit in multipliers:
```

```
    raw = value * multipliers[unit]
```

```
    if unit == "k":
```

```
        raw = max(1, raw // 1024)
```

```
    return raw
```

```
raise ValueError(f"Unknown size unit: {unit}")
```

```
def parse_duration_s(value_str: str, unit: str) -> int:
    """Parse a duration value + unit string into seconds.
```

```
>>> parse_duration_s("15", "m")
```

```
900
```

```
>>> parse_duration_s("2", "h")
```

```
7200
```

```
>>> parse_duration_s("90", "s")
```

```
90
```

```

Note: returns the raw arithmetic result; callers must enforce
``FilterConfig`` domain constraints (e.g. max >= min).
"""
value = int(value_str)
unit = unit.lower()
multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
if unit in multipliers:
    return value * multipliers[unit]
raise ValueError(f"Unknown time unit: {unit}")

```

--- formatting helpers -----

```

def _format_duration(seconds: int | None) -> str:
    """Format seconds as a human-readable duration string."""
    if seconds is None:
        return "unlimited"
    if seconds == 0:
        return "0s"
    if seconds < 60:
        return f"{seconds}s"
    if seconds < 3600:
        m, s = divmod(seconds, 60)
        return f"{m}m{s}s" if s else f"{m}m"
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    parts = f"{h}h"
    if m:
        parts += f"{m}m"
    if s:
        parts += f"{s}s"
    return parts

```

```

def _format_size(mb: int | None) -> str:
    """Format mebibytes as a human-readable size string."""
    if mb is None:
        return "unlimited"
    if mb == 0:
        return "0MB"
    if mb >= 1024:
        gb = mb / 1024
        if gb == int(gb):
            return f"{int(gb)}GB"
        return f"{gb:.1f}GB"
    return f"{mb}MB"

```

```

def format_config(cfg: FilterConfig) -> str:
    """Format a ``FilterConfig`` as a readable Telegram message."""
    lines = [
        "? **Current Filter Configuration**",
        "",
        f"? **Duration:** {_format_duration(cfg.min_duration_s)} - "
        f"{_format_duration(cfg.max_duration_s)}",
        f"? **Min resolution:** {' ' + str(cfg.target_height) + 'p' if cfg.target_height else "disabled"}",
        f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
        f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
        f"? **Delivery target:** `{cfg.delivery_target}`",
        "",
        "? **Content exclusions:**",
        f"    GIF/animated: {'? excluded' if cfg.exclude_gif else '? allowed'}",
    ]

```

```

        f"    Round videos: {'? excluded' if cfg.exclude_round else '? allowed'}",
        f"    Silent (no audio): {'? excluded' if cfg.exclude_nosound else '? allowed'}",
    ]
    return "\n".join(lines)

def format_commands() -> str:
    """Return a help message listing all available commands."""
    return "\n".join(
        [
            "? **Available Commands**",
            "",
            "? **Filter config**",
            "`/show config` ? Show current filter settings",
            "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
            "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing `B`",
            "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
            "    or `/set max length <N><unit>` ? same as above",
            "    Units: `s` (default), `m`, `h` ? case-insensitive",
            "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length 5s`)",
            "    Units: `s` (default), `m`, `h` ? case-insensitive",
            "`/set min resolution <N>p` ? Set min video height (e.g. `/set min resolution",
720p`)",
            "    or `/set min res <N>p` ? same as above",
            "",
            "? **Content exclusions** (all off by default)",
            "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
            "`/set exclude round on|off` ? Exclude circular video messages",
            "`/set exclude nosound on|off` ? Exclude videos without audio\n",
            "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs too.",
            "",
            "? **Backfill**",
            "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history (default",
90d)",
            "`/stop` ? Cancel running backfill",
            "",
            "\u2139\u2013 **Help**",
            "`/show commands` ? Show this help message",
        ]
    )
)

```

--- command handler factory -----

```

def make_config_command_handler(
    user: User,
    conn: aioredis.Connection,
    engine_holder: dict[str, FilterEngine],
) -> Callable:
    """Return an async handler for /set and /show config commands.

    The handler only responds to messages the user sends to themselves
    (Saved Messages). It persists config changes via ``db.save_filter_config``
    and hot-reloads the filter engine through ``engine_holder``.

    ``engine_holder`` is a mutable dict that both the live-message handler and
    the backfill handler read from, so a config change is picked up by all
    components immediately.
    """

    async def on_command(event: object) -> None:
        message = getattr(event, "message", None)
        if message is None:

```

```

        return

# Only respond to messages the user sent to themselves.
if not getattr(message, "out", False):
    return
peer = getattr(message, "peer_id", None)
peer_user_id = getattr(peer, "user_id", None) if peer else None
if peer_user_id != user.id:
    return

text: str = getattr(message, "message", "") or ""
# Strip trailing whitespace so mobile keyboards don't silently
# reject commands (e.g. "/show config " ? "/show config").
text = text.rstrip()
client: TelegramClient = getattr(event, "client", None) # type: ignore[assignment]

# --- /set size <value><unit> -----
m = _SIZE_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_mb = parse_size_mb(value_str, unit)
    except ValueError:
        await _reply(client, message, "? Invalid size format. Use e.g. `/set size
10MB`.")
        return
    if new_mb < 1:
        await _reply(
            client, message, "? Size must be at least 1 MB (`FilterConfig` constraint).")
        return
    cfg = await _update_config(conn, user.id, max_size_mb=new_mb)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max file size set to **{_format_size(new_mb)}**."
f"\n\n{format_config(cfg)}",
    )
    return

# --- /set length <value><unit> -----
m = _LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_secs = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client, message, "? Invalid duration format. Use e.g. `/set length 15m`."
        )
        return
    # Enforce max_duration_s >= min_duration_s so every video isn't
    # silently rejected (and so the FilterConfig validator passes on
    # next startup).
    current_cfg = await db.load_filter_config(conn, user.id)
    min_secs = current_cfg.min_duration_s or 0
    if new_secs < min_secs:
        await _reply(
            client,
            message,
            f"? Max duration ({_format_duration(new_secs)}) must be "
            f">= min duration ({_format_duration(min_secs)}).",
        )
        return

```

```

    cfg = await _update_config(conn, user.id, max_duration_s=new_secs)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Max duration set to **{_format_duration(new_secs)}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# --- /set min length <value><unit> -----
m = _MIN_LENGTH_RE.match(text)
if m:
    value_str, unit = m.group(1), m.group(2) or ""
    try:
        new_min = parse_duration_s(value_str, unit)
    except ValueError:
        await _reply(
            client,
            message,
            "? Invalid duration format. Use e.g. `/set min length 5s`.",
        )
        return
    # Enforce min_duration_s <= max_duration_s.
    current_cfg = await db.load_filter_config(conn, user.id)
    max_secs = current_cfg.max_duration_s or 0
    if max_secs > 0 and new_min > max_secs:
        await _reply(
            client,
            message,
            f"? Min duration ({_format_duration(new_min)}) must be "
            f"<= max duration ({_format_duration(max_secs)}).",
        )
        return
    cfg = await _update_config(conn, user.id, min_duration_s=new_min)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Min duration set to **{_format_duration(new_min)}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# --- /set min resolution <N>p -----
m = _MIN_RES_RE.match(text)
if m:
    new_height = int(m.group(1))
    # Pre-validate for a friendly message (FilterConfig also enforces ge=1).
    if new_height < 1:
        await _reply(client, message, "? Resolution must be at least 1p.")
        return
    cfg = await _update_config(conn, user.id, target_height=new_height)
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? Min resolution set to **{new_height}p**." f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for malformed /set min resolution|res.
if _MIN_RES_HELP_RE.match(text):
    await _reply(
        client,

```

```

        message,
        "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)",
    )
    return

# --- /set exclude <gif|round|nosound> <on|off> -----
m = _EXCLUDE_RE.match(text)
if m:
    what = m.group(1).lower()
    state = m.group(2).lower()
    enabled = state == "on"
    labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}}
    field_map = {
        "gif": "exclude_gif",
        "round": "exclude_round",
        "nosound": "exclude_nosound",
    }
    cfg = await _update_config(conn, user.id, **{field_map[what]: enabled})
    engine_holder[ENGINE_KEY] = FilterEngine(cfg)
    await _reply(
        client,
        message,
        f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
        f"\n\n{format_config(cfg)}",
    )
    return

# Friendly error for invalid /set exclude (e.g. "gif true", missing on/off).
if _EXCLUDE_HELP_RE.match(text):
    await _reply(
        client,
        message,
        "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
        "Example: `/set exclude gif on`",
    )
    return

# --- /show config -----
if _SHOW_CONFIG_RE.match(text):
    cfg = await db.load_filter_config(conn, user.id)
    await _reply(client, message, format_config(cfg))
    return

# --- /show commands -----
if _SHOW_COMMANDS_RE.match(text):
    await _reply(client, message, format_commands())
    return

return on_command

```

--- internal helpers -----

```

async def _update_config(
    conn: aiosqlite.Connection,
    user_id: int,
    max_size_mb: int | None = None,
    max_duration_s: int | None = None,
    min_duration_s: int | None = None,
    target_height: int | None = None,
    exclude_gif: bool | None = None,
    exclude_round: bool | None = None,

```



```

        exclude_nosound: bool | None = None,
    ) -> FilterConfig:
        """Load the current config, apply partial updates, persist, and return."""
        cfg = await db.load_filter_config(conn, user_id)
        if max_size_mb is not None:
            cfg = cfg.model_copy(update={"max_size_mb": max_size_mb})
        if max_duration_s is not None:
            cfg = cfg.model_copy(update={"max_duration_s": max_duration_s})
        if min_duration_s is not None:
            cfg = cfg.model_copy(update={"min_duration_s": min_duration_s})
        if target_height is not None:
            cfg = cfg.model_copy(update={"target_height": target_height})
        if exclude_gif is not None:
            cfg = cfg.model_copy(update={"exclude_gif": exclude_gif})
        if exclude_round is not None:
            cfg = cfg.model_copy(update={"exclude_round": exclude_round})
        if exclude_nosound is not None:
            cfg = cfg.model_copy(update={"exclude_nosound": exclude_nosound})
        await db.save_filter_config(conn, user_id, cfg)
        log.info(
            "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
            "target_height=%s "
            "exclude_gif=%s exclude_round=%s exclude_nosound=%s",
            user_id,
            cfg.max_size_mb,
            cfg.min_duration_s,
            cfg.max_duration_s,
            cfg.target_height,
            cfg.exclude_gif,
            cfg.exclude_round,
            cfg.exclude_nosound,
        )
    return cfg

```

```

async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
    """Send a reply, logging network errors but not raising.

```

```

    Only swallows network/RPC errors so programming bugs (KeyError,
    AttributeError, TypeError) still surface loudly during development.
    """

```

```

    try:
        await client.send_message("me", text)
    except (OSError, TimeoutError, RPCError):
        log.warning("config_reply_failed", exc_info=True)

```

```

__all__ = [
    "ENGINE_KEY",
    "_EXCLUDE_RE",
    "_LENGTH_RE",
    "_MIN_LENGTH_RE",
    "_MIN_RES_RE",
    "_MIN_RES_HELP_RE",
    "_SHOW_COMMANDS_RE",
    "_SHOW_CONFIG_RE",
    "_SIZE_RE",
    "_format_duration",
    "_format_size",
    "_update_config",
    "format_commands",
    "format_config",
    "make_config_command_handler",
    "parse_duration_s",
    "parse_size_mb",

```

]

4. user (2026-07-06T08:39:31.818Z)

```
1      """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3      Extracts command logic from the Telethon adapter (`commands.py`) so the
4      same `/set`, `/show` commands work identically through a Bot API
5      front-door. The service is pure: strings in, `CommandResult` out ? no
6      Telethon objects, no network calls.
7
8      See `docs/adr/0007-inline-config-commands.md` for the command model and
9      `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API motivation.
10     """
11
12     from __future__ import annotations
13
14     import logging
15     import re
16     from dataclasses import dataclass
17
18     import aiosqlite
19
20     from tg_video_filter import db
21     from tg_video_filter.models import FilterConfig
22
23     log = logging.getLogger(__name__)
24
25     # Key into the shared engine_holder dict (imported by commands.py's adapter).
26     ENGINE_KEY = "engine"
27
28     # --- regex patterns -----
29
30     _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmg]?b?)$", re.IGNORECASE)
31     _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)", re.IGNORECASE)
32     _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)", re.IGNORECASE)
33     _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
34     _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
35     _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
36     _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
37     _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
38     _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
39
40
41     # --- unit parsing -----
42
43
44     def parse_size_mb(value_str: str, unit: str) -> int:
45         """Parse a size value + unit string into mebibytes."""
46         value = int(value_str)
47         unit = unit.lower().rstrip("b")
48         multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
49         if unit in multipliers:
50             raw = value * multipliers[unit]
51             if unit == "k":
52                 raw = max(1, raw // 1024)
53             return raw
54         raise ValueError(f"Unknown size unit: {unit}")
55
56
57     def parse_duration_s(value_str: str, unit: str) -> int:
58         """Parse a duration value + unit string into seconds."""
59         value = int(value_str)
```

```

60     unit = unit.lower()
61     multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
62     if unit in multipliers:
63         return value * multipliers[unit]
64     raise ValueError(f"Unknown time unit: {unit}")
65
66
67 # --- formatting helpers -----
68
69
70 def _format_duration(seconds: int | None) -> str:
71     if seconds is None:
72         return "unlimited"
73     if seconds == 0:
74         return "0s"
75     if seconds < 60:
76         return f"{seconds}s"
77     if seconds < 3600:
78         m, s = divmod(seconds, 60)
79         return f"{m}m{s}s" if s else f"{m}m"
80     h, rem = divmod(seconds, 3600)
81     m, s = divmod(rem, 60)
82     parts = f"{h}h"
83     if m:
84         parts += f"{m}m"
85     if s:
86         parts += f"{s}s"
87     return parts
88
89
90 def _format_size(mb: int | None) -> str:
91     if mb is None:
92         return "unlimited"
93     if mb == 0:
94         return "0MB"
95     if mb >= 1024:
96         gb = mb / 1024
97         if gb == int(gb):
98             return f"{int(gb)}GB"
99         return f"{gb:.1f}GB"
100     return f"{mb}MB"
101
102
103 def format_config(cfg: FilterConfig) -> str:
104     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
105     lines = [
106         "? **Current Filter Configuration**",
107         "",
108         f"?? **Duration:** {_format_duration(cfg.min_duration_s)} -",
109         f"{_format_duration(cfg.max_duration_s)}",
110         f"? **Min resolution:** {'? ' + str(cfg.target_height) + 'p' if",
111         f"cfg.target_height else 'disabled'}",
112         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
113         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
114         f"? **Delivery target:** `{cfg.delivery_target}`",
115         "",
116         "? **Content exclusions:**",
117         f"    GIF/animated: {'? excluded' if cfg.exclude_gif else '? allowed'}",
118         f"    Round videos: {'? excluded' if cfg.exclude_round else '? allowed'}",
119         f"    Silent (no audio): {'? excluded' if cfg.exclude_nosound else '? allowed'}",
120     ]
121     return "\n".join(lines)
122
123 def format_commands() -> str:

```

```

123     """Return a help message listing all available commands."""
124     return "\n".join(
125         [
126             "? **Available Commands**",
127             "",
128             "? **Filter config**",
129             "`/show config` ? Show current filter settings",
130             "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
131             "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing
132             `B`",
133             "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
134             "    or `/set max length <N><unit>` ? same as above",
135             "    Units: `s` (default), `m`, `h` ? case-insensitive",
136             "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length
137             5s`)",
138             "    Units: `s` (default), `m`, `h` ? case-insensitive",
139             "`/set min resolution <N>p` ? Set min video height (e.g. `/set min
140             resolution 720p`)",
141             "    or `/set min res <N>p` ? same as above",
142             "",
143             "? **Content exclusions** (all off by default)",
144             "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
145             "`/set exclude round on|off` ? Exclude circular video messages",
146             "`/set exclude nosound on|off` ? Exclude videos without audio\n"
147             "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs
148             too.",
149             "",
150             "? **Backfill**",
151             "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
152             (default 90d)",
153             "`/stop` ? Cancel running backfill",
154             "",
155             "\u2139\u2013 **Help**",
156             "`/show commands` ? Show this help message",
157         ]
158     )
159
160 # --- command result -----
161
162 @dataclass
163 class CommandResult:
164     """The output of executing a command ? provider-independent.
165
166     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
167     and ``config_changed`` to decide whether to hot-reload the filter engine.
168     """
169
170     reply_text: str = ""
171     config_changed: bool = False
172
173 # --- command service -----
174
175 class CommandService:
176     """Provider-independent command execution.
177
178     Parses a command string against an account's current config and DB,
179     returning a ``CommandResult``. No Telethon objects involved ? pure string
180     in, effects out. The Telethon adapter (``commands.py``) handles the
181     transport (checking message.out, sending the reply); the Bot API adapter
182     (future) calls ``handle()`` with the ``from.id`` from the update.

```

```

183 Usage::
184
185     svc = CommandService()
186     result = await svc.handle(user_id, text, conn, current_cfg)
187     if result.config_changed:
188         engine_holder["engine"] = FilterEngine(current_cfg)
189     await send_reply(result.reply_text)
190
191
192 # -- public API -----
193
194 async def handle(
195     self,
196     user_id: int,
197     text: str,
198     conn: aiosqlite.Connection,
199     current_cfg: FilterConfig,
200 ) -> CommandResult:
201     """Parse and execute a command text, returning the reply + change flag."""
202     text = text.rstrip()
203
204     # --- /set size <value><unit> -----
205     m = _SIZE_RE.match(text)
206     if m:
207         value_str, unit = m.group(1), m.group(2) or ""
208         try:
209             new_mb = parse_size_mb(value_str, unit)
210         except ValueError:
211             return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
212         if new_mb < 1:
213             return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
214         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
215         return CommandResult(
216             f"? Max file size set to
**[_format_size(new_mb)]*.\\n\\n{format_config(cfg)}",
217             config_changed=True,
218         )
219
220     # --- /set length <value><unit> -----
221     m = _LENGTH_RE.match(text)
222     if m:
223         value_str, unit = m.group(1), m.group(2) or ""
224         try:
225             new_secs = parse_duration_s(value_str, unit)
226         except ValueError:
227             return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
228         min_secs = current_cfg.min_duration_s or 0
229         if new_secs < min_secs:
230             return CommandResult(
231                 f"? Max duration ({_format_duration(new_secs)}) must be "
232                 f"?>= min duration ({_format_duration(min_secs)})."
233             )
234         cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
235         return CommandResult(
236             f"? Max duration set to
**[_format_duration(new_secs)]*.\\n\\n{format_config(cfg)}",
237             config_changed=True,
238         )
239
240     # --- /set min length <value><unit> -----
241     m = _MIN_LENGTH_RE.match(text)

```

```

242         if m:
243             value_str, unit = m.group(1), m.group(2) or ""
244             try:
245                 new_min = parse_duration_s(value_str, unit)
246             except ValueError:
247                 return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
248             max_secs = current_cfg.max_duration_s or 0
249             if max_secs > 0 and new_min > max_secs:
250                 return CommandResult(
251                     f"? Min duration ({_format_duration(new_min)}) must be "
252                     f"<= max duration ({_format_duration(max_secs)})".
253                 )
254             cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
255             return CommandResult(
256                 f"? Min duration set to
**{_format_duration(new_min)}**.\n\n{format_config(cfg)}",
257                 config_changed=True,
258             )
259
260         # --- /set min resolution <N>p -----
261         m = _MIN_RES_RE.match(text)
262         if m:
263             new_height = int(m.group(1))
264             if new_height < 1:
265                 return CommandResult("? Resolution must be at least 1p.")
266             cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
267             return CommandResult(
268                 f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
269                 config_changed=True,
270             )
271
272         if _MIN_RES_HELP_RE.match(text):
273             return CommandResult(
274                 "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
275             )
276
277         # --- /set exclude <gif|round|nosound> <on|off> -----
278         m = _EXCLUDE_RE.match(text)
279         if m:
280             what = m.group(1).lower()
281             state = m.group(2).lower()
282             enabled = state == "on"
283             labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
284             field_map = {
285                 "gif": "exclude_gif",
286                 "round": "exclude_round",
287                 "nosound": "exclude_nosound",
288             }
289             cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
290             return CommandResult(
291                 f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
292                 f"\n\n{format_config(cfg)}",
293                 config_changed=True,
294             )
295
296         if _EXCLUDE_HELP_RE.match(text):
297             return CommandResult(
298                 "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
299                 "Example: `/set exclude gif on`"
300             )
301

```

```

302         # --- /show config -----
303         if _SHOW_CONFIG_RE.match(text):
304             cfg = await db.load_filter_config(conn, user_id)
305             return CommandResult(format_config(cfg))
306
307         # --- /show commands -----
308         if _SHOW_COMMANDS_RE.match(text):
309             return CommandResult(format_commands())
310
311         # Not a recognised command ? return empty result (noise).
312         return CommandResult()
313
314     # -- internal -----
315
316     async def _update(
317         self,
318         conn: aiosqlite.Connection,
319         user_id: int,
320         current: FilterConfig,
321         max_size_mb: int | None = None,
322         max_duration_s: int | None = None,
323         min_duration_s: int | None = None,
324         target_height: int | None = None,
325         exclude_gif: bool | None = None,
326         exclude_round: bool | None = None,
327         exclude_nosound: bool | None = None,
328     ) -> FilterConfig:
329         """Apply partial updates to the account's config, persist, and return."""
330         updates: dict[str, object] = {}
331         if max_size_mb is not None:
332             updates["max_size_mb"] = max_size_mb
333         if max_duration_s is not None:
334             updates["max_duration_s"] = max_duration_s
335         if min_duration_s is not None:
336             updates["min_duration_s"] = min_duration_s
337         if target_height is not None:
338             updates["target_height"] = target_height
339         if exclude_gif is not None:
340             updates["exclude_gif"] = exclude_gif
341         if exclude_round is not None:
342             updates["exclude_round"] = exclude_round
343         if exclude_nosound is not None:
344             updates["exclude_nosound"] = exclude_nosound
345         cfg = current.model_copy(update=updates) if updates else current
346         await db.save_filter_config(conn, user_id, cfg)
347         log.info(
348             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
349             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
350             user_id,
351             cfg.max_size_mb,
352             cfg.min_duration_s,
353             cfg.max_duration_s,
354             cfg.target_height,
355             cfg.exclude_gif,
356             cfg.exclude_round,
357             cfg.exclude_nosound,
358         )
359         return cfg
360
361     __all__ = [
362         "ENGINE_KEY",
363         "CommandResult",
364         "CommandService",
365         "_format_duration",

```

```

367         "_format_size",
368         "format_config",
369         "format_commands",
370         "parse_duration_s",
371         "parse_size_mb",
372     ]
373

```

5. user (2026-07-06T08:39:32.000Z)

```

1     """Config commands ? Telethon adapter over the provider-independent `CommandService`.
2
3     This module is a thin Telethon adapter. It handles Telethon-specific transport
4     (checking `message.out`, `peer_id`, sending replies) and delegates command
5     parsing and config mutation to `command_service.CommandService` (ADR 0011, Track D).
6
7     Re-exports regex patterns and unit-parsers for backwards-compatibility with
8     existing tests (`tests/test_commands.py`).
9
10    See `docs/adr/0007-inline-config-commands.md` for the command model and
11    `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API front-door.
12    """
13
14    from __future__ import annotations
15
16    import logging
17    from collections.abc import Callable
18
19    import aiosqlite
20    from telethon import TelegramClient
21    from telethon.errors import RPCError
22
23    from tg_video_filter import db
24    from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25    from tg_video_filter.filters import FilterEngine
26    from tg_video_filter.models import FilterConfig, User
27
28    log = logging.getLogger(__name__)
29
30    from tg_video_filter.command_service import ( # noqa: E402
31        _EXCLUDE_HELP_RE,
32        _EXCLUDE_RE,
33        _LENGTH_RE,
34        _MIN_LENGTH_RE,
35        _MIN_RES_HELP_RE,
36        _MIN_RES_RE,
37        _SHOW_COMMANDS_RE,
38        _SHOW_CONFIG_RE,
39        _SIZE_RE,
40        _format_duration,
41        _format_size,
42        format_commands,
43        format_config,
44        parse_duration_s,
45        parse_size_mb,
46    )
47
48
49    def make_config_command_handler(
50        user: User,
51        conn: aiosqlite.Connection,
52        engine_holder: dict[str, FilterEngine],
53    ) -> Callable:
54        svc = CommandService()
55

```



```

56     async def on_command(event: object) -> None:
57         message = getattr(event, "message", None)
58         if message is None:
59             return
60         if not getattr(message, "out", False):
61             return
62         peer = getattr(message, "peer_id", None)
63         peer_user_id = getattr(peer, "user_id", None) if peer else None
64         if peer_user_id != user.id:
65             return
66
67         text: str = getattr(message, "message", "") or ""
68         client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
69
70         current_cfg = await db.load_filter_config(conn, user.id)
71         result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
72
73         if result.config_changed:
74             new_cfg = await db.load_filter_config(conn, user.id)
75             engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
76
77         if result.reply_text:
78             await _reply(client, message, result.reply_text)
79
80     return on_command
81
82
83 # --- backwards-compat helpers (test_commands.py references) -----
84
85
86 async def _update_config(
87     conn: aiosqlite.Connection,
88     user_id: int,
89     max_size_mb: int | None = None,
90     max_duration_s: int | None = None,
91     min_duration_s: int | None = None,
92     target_height: int | None = None,
93     exclude_gif: bool | None = None,
94     exclude_round: bool | None = None,
95     exclude_nosound: bool | None = None,
96 ) -> FilterConfig:
97     """Deprecated wrapper ? delegates to ``CommandService._update``.
98
99     Kept for backwards-compatibility with ``tests/test_commands.py`` which
100     imports ``_update_config`` from here. New code should call
101     :meth:`CommandService.handle` which returns a :class:`CommandResult`.
102     """
103     svc = CommandService()
104     cfg = await db.load_filter_config(conn, user_id)
105     return await svc._update(
106         conn,
107         user_id,
108         cfg,
109         max_size_mb=max_size_mb,
110         max_duration_s=max_duration_s,
111         min_duration_s=min_duration_s,
112         target_height=target_height,
113         exclude_gif=exclude_gif,
114         exclude_round=exclude_round,
115         exclude_nosound=exclude_nosound,
116     )
117
118
119 async def _reply(client: TelegramClient, to_message: object, text: str) -> None:

```

```

120         try:
121             await client.send_message("me", text)
122         except (OSError, TimeoutError, RPCError):
123             log.warning("config_reply_failed", exc_info=True)
124
125
126     __all__ = [
127         "ENGINE_KEY",
128         "_EXCLUDE_RE",
129         "_LENGTH_RE",
130         "_MIN_LENGTH_RE",
131         "_MIN_RES_RE",
132         "_SHOW_CONFIG_RE",
133         "_SHOW_COMMANDS_RE",
134         "_EXCLUDE_HELP_RE",
135         "_MIN_RES_HELP_RE",
136         "_SIZE_RE",
137         "_format_duration",
138         "_format_size",
139         "_update_config",
140         "format_commands",
141         "format_config",
142         "make_config_command_handler",
143         "parse_duration_s",
144         "parse_size_mb",
145     ]
146

```

6. assistant (2026-07-06T08:39:36.355Z)

Now let me look at the test file and set up a differential repro. Let me first check the db module and models to build a proper repro.

7. user (2026-07-06T08:39:37.389Z)

```

1     """Tests for the provider-independent CommandService (command_service.py).
2
3     These tests exercise command parsing and config mutation without any
4     Telethon objects ? pure strings in, CommandResult out. The Telethon
5     adapter (commands.py) is tested separately via test_commands.py.
6     """
7
8     from __future__ import annotations
9
10    import aiosqlite
11    import pytest
12
13    from tg_video_filter import db
14    from tg_video_filter.command_service import (
15        CommandService,
16        format_commands,
17        parse_duration_s,
18        parse_size_mb,
19    )
20    from tg_video_filter.models import FilterConfig
21
22    USER_ID = 111
23
24
25    @pytest.fixture
26    async def conn() -> aiosqlite.Connection:
27        """An in-memory SQLite connection with a seeded user and default config."""
28        c = await aiosqlite.connect(":memory:")
29        await db._apply_schema(c)
30        await db.upsert_user(

```

```

31         c,
32         db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
33     )
34     await db.save_filter_config(c, USER_ID, FilterConfig())
35     yield c
36     await c.close()
37
38
39 @pytest.fixture
40 def svc() -> CommandService:
41     return CommandService()
42
43
44 # --- unit parser tests -----
45
46
47 @pytest.mark.parametrize(
48     "value, unit, expected",
49     [
50         ("10", "MB", 10),
51         ("1", "GB", 1024),
52         ("500", "KB", 1),
53         ("5", "", 5),
54         ("2", "TB", 2 * 1024 * 1024),
55     ],
56 )
57 def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
58     assert parse_size_mb(value, unit) == expected
59
60
61 def test_parse_size_mb_unknown_unit_raises() -> None:
62     with pytest.raises(ValueError):
63         parse_size_mb("10", "XB")
64
65
66 @pytest.mark.parametrize(
67     "value, unit, expected",
68     [
69         ("15", "m", 900),
70         ("2", "h", 7200),
71         ("90", "s", 90),
72         ("5", "", 5),
73     ],
74 )
75 def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
76     assert parse_duration_s(value, unit) == expected
77
78
79 def test_parse_duration_s_unknown_unit_raises() -> None:
80     with pytest.raises(ValueError):
81         parse_duration_s("10", "x")
82
83
84 # --- /set size -----
85
86
87 async def test_set_size(conn: aiomysql.Connection, svc: CommandService) -> None:
88     result = await svc.handle(USER_ID, "/set size 100MB", conn, FilterConfig())
89     assert "100MB" in result.reply_text
90     assert result.config_changed is True
91
92
93 async def test_set_size_invalid_format_returns_empty(
94     conn: aiomysql.Connection, svc: CommandService
95 ) -> None:

```

```

96         """Unparseable input that doesn't match the regex returns empty result (silent)."""
97         result = await svc.handle(USER_ID, "/set size not-a-number", conn, FilterConfig())
98         assert result.reply_text == "" # regex doesn't match, no reply
99         assert result.config_changed is False
100
101
102     async def test_set_size_below_minimum(conn: aiomysql.Connection, svc: CommandService)
-> None:
103         result = await svc.handle(USER_ID, "/set size 0MB", conn, FilterConfig())
104         assert "at least 1 MB" in result.reply_text
105         assert result.config_changed is False
106
107
108     # --- /set length -----
109
110
111     async def test_set_length(conn: aiomysql.Connection, svc: CommandService) -> None:
112         result = await svc.handle(USER_ID, "/set length 15m", conn, FilterConfig())
113         assert "15m" in result.reply_text or "900s" in result.reply_text
114         assert result.config_changed is True
115
116
117     async def test_set_length_below_min(conn: aiomysql.Connection, svc: CommandService) ->
None:
118         cfg = FilterConfig(min_duration_s=60)
119         result = await svc.handle(USER_ID, "/set length 30s", conn, cfg)
120         assert "must be >= min duration" in result.reply_text
121         assert result.config_changed is False
122
123
124     # --- /set min length -----
125
126
127     async def test_set_min_length(conn: aiomysql.Connection, svc: CommandService) -> None:
128         result = await svc.handle(USER_ID, "/set min length 5s", conn, FilterConfig())
129         assert "5s" in result.reply_text
130         assert result.config_changed is True
131
132
133     async def test_set_min_length_above_max(conn: aiomysql.Connection, svc: CommandService)
-> None:
134         cfg = FilterConfig(max_duration_s=10)
135         result = await svc.handle(USER_ID, "/set min length 60s", conn, cfg)
136         assert "must be <= max duration" in result.reply_text
137         assert result.config_changed is False
138
139
140     # --- /set min resolution -----
141
142
143     async def test_set_min_resolution(conn: aiomysql.Connection, svc: CommandService) ->
None:
144         result = await svc.handle(USER_ID, "/set min resolution 720p", conn, FilterConfig())
145         assert "720p" in result.reply_text
146         assert result.config_changed is True
147
148
149     async def test_set_min_resolution_below_min(
150         conn: aiomysql.Connection, svc: CommandService
151     ) -> None:
152         result = await svc.handle(USER_ID, "/set min resolution 0p", conn, FilterConfig())
153         assert "at least 1p" in result.reply_text
154         assert result.config_changed is False
155
156

```

```

157     async def test_set_min_res_help_on_malformed(
158         conn: aiomysqlite.Connection, svc: CommandService
159     ) -> None:
160         result = await svc.handle(USER_ID, "/set min resolution abc", conn, FilterConfig())
161         assert "Usage:" in result.reply_text
162
163
164     # --- /set exclude -----
165
166
167     async def test_set_exclude_gif_on(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
168         result = await svc.handle(USER_ID, "/set exclude gif on", conn, FilterConfig())
169         assert "ON" in result.reply_text
170         assert result.config_changed is True
171
172
173     async def test_set_exclude_round_off(conn: aiomysqlite.Connection, svc: CommandService) ->
None:
174         result = await svc.handle(USER_ID, "/set exclude round off", conn, FilterConfig())
175         assert "OFF" in result.reply_text
176         assert result.config_changed is True
177
178
179     async def test_set_exclude_help_on_malformed(
180         conn: aiomysqlite.Connection, svc: CommandService
181     ) -> None:
182         result = await svc.handle(USER_ID, "/set exclude gif true", conn, FilterConfig())
183         assert "Usage:" in result.reply_text
184         assert result.config_changed is False
185
186
187     # --- /show config -----
188
189
190     async def test_show_config(conn: aiomysqlite.Connection, svc: CommandService) -> None:
191         result = await svc.handle(USER_ID, "/show config", conn, FilterConfig())
192         assert "Current Filter Configuration" in result.reply_text
193         assert result.config_changed is False
194
195
196     # --- /show commands -----
197
198
199     async def test_show_commands(conn: aiomysqlite.Connection, svc: CommandService) -> None:
200         result = await svc.handle(USER_ID, "/show commands", conn, FilterConfig())
201         assert "Available Commands" in result.reply_text
202         assert result.config_changed is False
203
204
205     # --- unrecognized command -----
206
207
208     async def test_unrecognized_returns_empty(conn: aiomysqlite.Connection, svc:
CommandService) -> None:
209         result = await svc.handle(USER_ID, "hello world", conn, FilterConfig())
210         assert result.reply_text == ""
211         assert result.config_changed is False
212
213
214     # --- format_commands -----
215
216
217     def test_format_commands_includes_all_sections() -> None:
218         text = format_commands()

```

```
219         assert "/show config" in text
220         assert "/set size" in text
221         assert "/set exclude" in text
222         assert "/backfill" in text
223
```

8. user (2026-07-06T08:39:38.506Z)

```
199:async def _apply_schema(conn: aiomysqlite.Connection) -> None:
1634:async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1647:async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg: FilterConfig)
-> None:
```

9. assistant (2026-07-06T08:39:46.818Z)

Now I have a clear picture. The key architectural difference the prompt flags is the `current\_cfg`-vs-DB-reload divergence. Let me analyze this carefully.

Old code: Each branch that mutates calls `\_update\_config`, which *itself* reloads `cfg` = await db.load_filter_config(conn, user_id) fresh. And the length/min-length validators *also* independently reload `current\_cfg` = await db.load_filter_config(conn, user.id).

New code: The adapter loads `current\_cfg` once, passes it to `handle()`. The validators use that passed `current\_cfg`. `\_update` applies updates to that same passed `current` object (does NOT reload).

Within a single command invocation these are equivalent because nothing else mutates the config between the loads. Let me verify the `\_show config` path and the empty-`text` edge, plus run a differential battery. Let me check `load\_filter\_config` behavior for a non-existent config.

10. user (2026-07-06T08:39:48.024Z)

```
1634     async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1635         """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1636
1637         ADR 0011 (Option 2A): ``filter_config`` is renamed to
1638         ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1639         instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1640         """
1641         policy = await get_default_policy(conn, user_id)
1642         if policy is None:
1643             return FilterConfig()
1644         return FilterConfig.model_validate_json(policy.config_json)
1645
1646
1647     async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1648         """Deprecated shim: upsert the account's default :class:`Policy`."""
1649         existing = await get_default_policy(conn, user_id)
1650         await upsert_policy(
1651             conn,
1652             Policy(
1653                 id=existing.id if existing is not None else None,
1654                 account_id=user_id,
1655                 name="default",
1656                 config_json=cfg.model_dump_json(),
1657                 is_default=True,
1658                 created_at=(existing.created_at if existing is not None else _now()),
1659                 updated_at=_now(),
1660             ),
1661         )
1662
1663
1664     # --- bulk helpers (used by the manager on startup) -----
```

```

1665
1666
1667     async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1668         conn: aiosqlite.Connection, user_id: int, limit: int = 100
1669     ) -> list[VideoRecord]:
1670         """Deprecated shim: list the account's content items projected onto VideoRecord."""
1671         conn.row_factory = aiosqlite.Row
1672         async with conn.execute(
1673             """SELECT ci.*, s.provider_source_id AS src_pid

```

11. assistant (2026-07-06T08:39:53.150Z)

Now let me write a differential repro that reimplements the OLD handler logic and compares against the new `CommandService.handle()` across a battery of inputs, diffing both reply text and saved config.

12. assistant (2026-07-06T08:40:24.606Z)

```

"""Differential test: OLD inline handler logic vs NEW CommandService.handle.

```

```

Reimplements the OLD commands.py branch logic (from `git show main`) inline,
running it against the same in-memory DB semantics, and diffs reply_text +
resulting saved FilterConfig against the NEW CommandService.
"""

```

```

from __future__ import annotations

```

```

import asyncio

```

```

import aiosqlite

```

```

from tg_video_filter import db
from tg_video_filter.command_service import (
    CommandService,
    _EXCLUDE_HELP_RE,
    _EXCLUDE_RE,
    _LENGTH_RE,
    _MIN_LENGTH_RE,
    _MIN_RES_HELP_RE,
    _MIN_RES_RE,
    _SHOW_COMMANDS_RE,
    _SHOW_CONFIG_RE,
    _SIZE_RE,
    _format_duration,
    _format_size,
    format_commands,
    format_config,
    parse_duration_s,
    parse_size_mb,
)
from tg_video_filter.models import FilterConfig

```

```

USER_ID = 111

```

```

async def old_update_config(conn, user_id, **kwargs) -> FilterConfig:
    """Old_update_config: RELOADS from db each call."""
    cfg = await db.load_filter_config(conn, user_id)
    for k, v in kwargs.items():
        if v is not None:
            cfg = cfg.model_copy(update={k: v})
    await db.save_filter_config(conn, user_id, cfg)
    return cfg

```

```

async def old_handle(conn, user_id, text) -> str:
    """Reimplements OLD on_command branch logic. Returns reply_text ('' if none)."""
    text = text.rstrip()

    m = _SIZE_RE.match(text)
    if m:
        value_str, unit = m.group(1), m.group(2) or ""
        try:
            new_mb = parse_size_mb(value_str, unit)
        except ValueError:
            return "? Invalid size format. Use e.g. `/set size 10MB`."
        if new_mb < 1:
            return "? Size must be at least 1 MB (`FilterConfig` constraint).\"
        cfg = await old_update_config(conn, user_id, max_size_mb=new_mb)
        return f"? Max file size set to **{_format_size(new_mb)}**.\" f\"\\n\\n{format_config(cfg)}\"

    m = _LENGTH_RE.match(text)
    if m:
        value_str, unit = m.group(1), m.group(2) or ""
        try:
            new_secs = parse_duration_s(value_str, unit)
        except ValueError:
            return "? Invalid duration format. Use e.g. `/set length 15m`.\"
        current_cfg = await db.load_filter_config(conn, user_id)
        min_secs = current_cfg.min_duration_s or 0
        if new_secs < min_secs:
            return (
                f"? Max duration ({_format_duration(new_secs)}) must be \"
                f">= min duration ({_format_duration(min_secs)}).\"
            )
        cfg = await old_update_config(conn, user_id, max_duration_s=new_secs)
        return f"? Max duration set to **{_format_duration(new_secs)}**.\"
    f\"\\n\\n{format_config(cfg)}\"

    m = _MIN_LENGTH_RE.match(text)
    if m:
        value_str, unit = m.group(1), m.group(2) or ""
        try:
            new_min = parse_duration_s(value_str, unit)
        except ValueError:
            return "? Invalid duration format. Use e.g. `/set min length 5s`.\"
        current_cfg = await db.load_filter_config(conn, user_id)
        max_secs = current_cfg.max_duration_s or 0
        if max_secs > 0 and new_min > max_secs:
            return (
                f"? Min duration ({_format_duration(new_min)}) must be \"
                f"<= max duration ({_format_duration(max_secs)}).\"
            )
        cfg = await old_update_config(conn, user_id, min_duration_s=new_min)
        return f"? Min duration set to **{_format_duration(new_min)}**.\"
    f\"\\n\\n{format_config(cfg)}\"

    m = _MIN_RES_RE.match(text)
    if m:
        new_height = int(m.group(1))
        if new_height < 1:
            return "? Resolution must be at least 1p.\"
        cfg = await old_update_config(conn, user_id, target_height=new_height)
        return f"? Min resolution set to **{new_height}p**.\" f\"\\n\\n{format_config(cfg)}\"

    if _MIN_RES_HELP_RE.match(text):
        return "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)\

    m = _EXCLUDE_RE.match(text)
    if m:

```



```

        what = m.group(1).lower()
        state = m.group(2).lower()
        enabled = state == "on"
        labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent videos"}
        field_map = {"gif": "exclude_gif", "round": "exclude_round", "nosound":
"exclude_nosound"}
        cfg = await old_update_config(conn, user_id, **{field_map[what]: enabled})
        return (
            f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
f"\n\n{format_config(cfg)}"
        )

```

```

if _EXCLUDE_HELP_RE.match(text):
    return (
        "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
        "Example: `/set exclude gif on`"
    )

```

```

if _SHOW_CONFIG_RE.match(text):
    cfg = await db.load_filter_config(conn, user_id)
    return format_config(cfg)

```

```

if _SHOW_COMMANDS_RE.match(text):
    return format_commands()

```

```

return ""

```

```

async def make_conn():
    c = await aiosqlite.connect(":memory:")
    await db._apply_schema(c)
    await db.upsert_user(c, db.User(id=USER_ID, phone="+15555550100"))
    await db.save_filter_config(c, USER_ID, FilterConfig())
    return c

```

```

BATTERY = [
    "/set size 10MB",
    "/set size 1GB",
    "/set size 500KB",
    "/set size 0MB",
    "/set size 5",
    "/set size 2TB",
    "/set size 3xb",
    "/SET SIZE 10mb",
    "/set size 10MB ",
    "/set length 15m",
    "/set max length 15m",
    "/set length 2h",
    "/set length 90s",
    "/set length 5",
    "/set min length 5s",
    "/set min length 10m",
    "/set min resolution 720p",
    "/set min res 480p",
    "/set min resolution 0p",
    "/set min resolution abc",
    "/set min res xyz",
    "/set exclude gif on",
    "/set exclude round off",
    "/set exclude nosound on",
    "/set exclude gif true",
    "/set exclude foo on",
    "/show config",
    "/show commands",

```

```

"/SHOW CONFIG",
"hello world",
"",
" ",
"/set length 30s", # with min preset below
"/set min length 60s", # with max preset below
]

```

```

async def run_case(text, seed=None):
    old_conn = await make_conn()
    new_conn = await make_conn()
    if seed:
        await db.save_filter_config(old_conn, USER_ID, seed)
        await db.save_filter_config(new_conn, USER_ID, seed)

    old_reply = await old_handle(old_conn, USER_ID, text)
    old_cfg = await db.load_filter_config(old_conn, USER_ID)

    svc = CommandService()
    new_current = await db.load_filter_config(new_conn, USER_ID)
    result = await svc.handle(USER_ID, text, new_conn, new_current)
    new_reply = result.reply_text
    new_cfg = await db.load_filter_config(new_conn, USER_ID)

    await old_conn.close()
    await new_conn.close()

    divergences = []
    if old_reply != new_reply:
        divergences.append(f"REPLY DIFF:\n OLD={old_reply!r}\n NEW={new_reply!r}")
    if old_cfg.model_dump() != new_cfg.model_dump():
        divergences.append(f"CFG DIFF:\n OLD={old_cfg.model_dump()}\n
NEW={new_cfg.model_dump()}")
    return divergences

```

```

async def main():
    any_diff = False
    for text in BATTERY:
        divs = await run_case(text)
        if divs:
            any_diff = True
            print(f"=== INPUT {text!r} ===")
            for d in divs:
                print(d)
    # Seeded cases
    seeds = [
        ("/set length 30s", FilterConfig(min_duration_s=60)),
        ("/set min length 60s", FilterConfig(max_duration_s=10)),
        ("/set length 30s", FilterConfig(min_duration_s=30)),
        ("/set min length 10s", FilterConfig(max_duration_s=10)),
    ]
    for text, seed in seeds:
        divs = await run_case(text, seed)
        if divs:
            any_diff = True
            print(f"=== SEEDED INPUT {text!r} seed={seed.model_dump()} ===")
            for d in divs:
                print(d)

    print("\nDONE. Divergence found:" , any_diff)

```

```

asyncio.run(main())

```

13. user (2026-07-06T08:40:24.632Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/diff_repro.py (file state is current in your context ? no need to Read it back)

14. user (2026-07-06T08:40:31.426Z)

DONE. Divergence found: False

15. assistant (2026-07-06T08:40:36.160Z)

No divergence across the full battery, including seeded validator cases, case-insensitivity, trailing whitespace, malformed inputs, and all unit variants. Both reply strings and saved configs are identical.

Now let me verify the remaining non-parsing concerns the prompt flagged: the adapter's message gate, re-export completeness, and the engine-rebuild path. Let me confirm the `\_\_all\_\_` re-exports match and check what `test\_commands.py` imports.

16. user (2026-07-06T08:40:40.299Z)

--- OLD __all__ symbols referenced ---

```
"_EXCLUDE_RE"
"_format_duration"
"_format_size"
"_LENGTH_RE"
"_MIN_LENGTH_RE"
"_MIN_RES_HELP_RE"
"_MIN_RES_RE"
"_SHOW_COMMANDS_RE"
"_SHOW_CONFIG_RE"
"_SIZE_RE"
"_update_config"
"b"
"client"
"config_reply_failed"
"engine"
"ENGINE_KEY"
"exclude_gif"
"exclude_nosound"
"exclude_round"
"format_commands"
"format_config"
"g"
"GB"
"gif"
"h"
"k"
"KB"
"m"
"make_config_command_handler"
"max_duration_s"
"max_size_mb"
"MB"
"me"
"message"
"min_duration_s"
"nosound"
"on"
"out"
"parse_duration_s"
"parse_size_mb"
"peer_id"
```

```

"round"
"s"
"t"
"target_height"
"unlimited"
"user_id"
=== test_commands.py imports from commands ===
17:from tg_video_filter.commands import (

```

17. user (2026-07-06T08:40:45.575Z)

```

17     from tg_video_filter.commands import (
18         _EXCLUDE_RE,
19         _LENGTH_RE,
20         _MIN_LENGTH_RE,
21         _MIN_RES_HELP_RE,
22         _MIN_RES_RE,
23         _SHOW_COMMANDS_RE,
24         _SHOW_CONFIG_RE,
25         _SIZE_RE,
26         ENGINE_KEY,
27         _format_duration,
28         _format_size,
29         _update_config,
30         format_commands,
31         format_config,
32         make_config_command_handler,
33         parse_duration_s,
34         parse_size_mb,
35     )
36     from tg_video_filter.filters import FilterEngine
37     from tg_video_filter.models import DeliveryMode, FilterConfig, User
38
39     # --- parse_size_mb -----
40
41
42     @pytest.mark.parametrize(
43         ("value", "unit", "expected"),
44         [
45             ("10", "MB", 10),
46             ("10", "mb", 10),
47             ("10", "M", 10),
48             ("10", "m", 10),
49             ("10", "", 10),
50             ("1", "GB", 1024),
51             ("1", "gb", 1024),
52             ("1", "G", 1024),
53             ("1", "g", 1024),
54             ("500", "KB", 1), # 500 KiB ? 0.49 MiB, floored to 1
55             ("1024", "KB", 1),
56             ("2048", "KB", 2),

```

18. assistant (2026-07-06T08:40:55.018Z)

All symbols imported by `test\_commands.py` are re-exported by the new `commands.py`. All confirmed present in the new `\_\_all\_\_` and the explicit re-import block (lines 30-46 of new commands.py). The `\_EXCLUDE\_HELP\_RE` re-export note: it's in `\_\_all\_\_` and re-imported too.

Now the key behavioral gate ? the adapter's message.out/peer_id gate and reply target. Let me confirm the old vs new adapter gate is identical, and check one subtle point: **the old adapter loaded `text` and applied `.rstrip()` on it, then the empty-text/noise path sent no reply. The new adapter passes raw `text` to `handle()` which returns empty `reply\_text` for noise, and only sends if `reply\_text` is truthy.** But there's one edge: in the OLD code, unrecognized commands simply fell through the handler with NO reply sent. In the NEW code, `handle()` returns `CommandResult()` (empty) and the adapter guards `if result.reply\_text:`. Equivalent.

Let me verify one more subtle thing: the `\_reply` call. Old code called `\_reply(client, message, ...)`_. New adapter calls `\_reply(client, message, result.reply\_text)`_. The `to\_message` param is unused in both. Equivalent.

Let me double-check the engine rebuild equivalence and whether `config\_changed` correctly gates it, including the edge where `/show config` and help messages must NOT rebuild the engine.

19. user (2026-07-06T08:41:03.171Z)

OK	'/set size 10MB'	config_changed=True
OK	'/set length 15m'	config_changed=True
OK	'/set min length 5s'	config_changed=True
OK	'/set min resolution 720p'	config_changed=True
OK	'/set exclude gif on'	config_changed=True
OK	'/set size 0MB'	config_changed=False
OK	'/set length 30s'	config_changed=False
OK	'/set min resolution abc'	config_changed=False
OK	'/set exclude gif true'	config_changed=False
OK	'/show config'	config_changed=False
OK	'/show commands'	config_changed=False
OK	'noise'	config_changed=False

20. assistant (2026-07-06T08:41:09.511Z)

All `config\_changed` flags correctly match the branches where the OLD code rebuilt the engine (only the 5 successful-mutation branches). Validation failures, help fallbacks, `/show`, and noise do not trigger a rebuild ? identical to old behavior.

Now let me examine the one genuine semantic difference the prompt flagged in detail: the **engine rebuild source config**. In the OLD code, the engine was built from the `cfg` returned by `\_update\_config` (the newly-saved config). In the NEW adapter (commands.py:73-75), on `config\_changed`, it does a **fresh** `db.load\_filter\_config` reload and builds `FilterEngine(new\_cfg)`. Since `\_update` already persisted via `save\_filter\_config`, the reload returns the same config. Let me confirm round-trip fidelity (save ? load produces identical config, no lossy serialization).

21. user (2026-07-06T08:41:16.823Z)

OK	'/set size 777MB'	reload==reply-embedded=True
OK	'/set length 3h'	reload==reply-embedded=True
OK	'/set min length 45s'	reload==reply-embedded=True
OK	'/set min resolution 1080p'	reload==reply-embedded=True
OK	'/set exclude nosound on'	reload==reply-embedded=True
OK	'/set exclude round on'	reload==reply-embedded=True

22. assistant (2026-07-06T08:41:24.751Z)

The reloaded config (new engine source) round-trips identically to the config embedded in the reply (which `\_update` returned ? the old engine source). Save/load is lossless, so `FilterEngine(reloaded) == FilterEngine(returned)`.

I've now verified every dimension flagged. Let me summarize the analysis and report no findings. Let me do one final check on the `current\_cfg` reload-vs-passed difference for a theoretical concurrency edge, and confirm the adapter's double-load is harmless.

The one architectural nuance worth stating precisely:

- **OLD**: `\_update\_config` reloaded from DB internally; validators (length/min-length) also independently reloaded. So within one command, old code did up to 2 DB reads.
- **NEW**: adapter loads `current\_cfg` once, passes it. `handle()` uses that single snapshot for validators AND as the base for `\_update`'s `model\_copy`. This is actually **more** internally consistent (single snapshot vs old's two independent reads), but since nothing mutates the config mid-command in the single-threaded asyncio dispatch, the results are identical. No divergence.

There is no behavioral difference. Let me record the final structured output.

23. user (2026-07-06T08:41:24.851Z)

Structured output provided successfully